EURASIP Journal on Wireless Communications and Networking

RESEARCH Open Access

Dynamic task scheduling in wireless edge computing using deep reinforcement learning with ordinal optimization

Yongyu Wang[1] and Siqi Sun[2*]

*Correspondence:
sunsiqi@bupt.edu.cn

[1] School of Information and Electronic Engineering, Shandong Technology and Business University, Yantai, China
[2] The State Key Laboratory of Networking and Switching Technology, Beijing University of Posts and Telecommunications, Beijing, China

Abstract

In latency-critical Internet of Things (IoT) applications, multi-access edge computing (MEC) in wireless networks reduces core network strain by pushing computation and data resources to the edge. However, the limited computing power of edge servers struggles to handle numerous computation-intensive tasks. To improve the quality of service (QoS) for IoT users, task scheduling in wireless edge computing to effectively allocate the computing resource becomes particularly urgent. The traditional game theory-based or auction-based scheduling approaches cannot adjust policies in dynamically changing environments. Moreover, the high-dimensional action space of the current deep reinforcement learning (DRL)-based methods causes them to converge slowly. To address this, we propose a novel deep reinforcement learning-based task scheduling algorithm that leverages ordinal optimization to enhance convergence speed and learning efficiency. The proposed method enables dynamically adaptive scheduling of tasks and computing resources across cloud centers and edge servers, aiming to minimize average service latency and energy consumption in IoT wireless edge networks. By integrating ordinal optimization with DRL, we achieve an improved tradeoff between exploration and exploitation, maximizing long-term utility in dynamic environments. Simulation results demonstrate that our approach significantly reduces latency and energy use compared to conventional and state-of-the-art techniques, with rapid convergence suitable for real-time applications.

Keywords: Wireless edge computing, IoT scheduling, Deep reinforcement learning, Ordinal optimization, Quality of service

1 Introduction

Due to the fast advancement of information technology, conventional cloud computing requires cloud data center to gather and process a substantial volume of data. This mode will lead to a long delay, low resources utilization, and a huge network traffic burden. Edge intelligence can bring learning intelligence from data centers to the network's edge [1]. As it is widely used, new challenges such as training efficiency and quality of service (QoS) emerge. With limited computing ability on network's edge, the QoS often cannot meet the expectations. To improve the QoS for end users, the cooperation between

Springer Open

© The Author(s) 2025. **Open Access** This article is licensed under a Creative Commons Attribution 4.0 International License, which permits use, sharing, adaptation, distribution and reproduction in any medium or format, as long as you give appropriate credit to the original author(s) and the source, provide a link to the Creative Commons licence, and indicate if changes were made. The images or other third party material in this article are included in the article's Creative Commons licence, unless indicated otherwise in a credit line to the material. If material is not included in the article's Creative Commons licence and your intended use is not permitted by statutory regulation or exceeds the permitted use, you will need to obtain permission directly from the copyright holder. To view a copy of this licence, visit http://creativecommons.org/licenses/by/4.0/.

edge computing and cloud computing to effectively schedule the computing resource becomes particularly urgent [2].

How to dynamically allocate computing and network resources in edge computing scenarios has become a great challenge. The scheduling or offloading of edge tasks can be expressed as a maximization or minimization problem with certain limitations, like network bandwidth or computing resources. A particularly helpful mathematical tool for studying optimization strategies is game theory. In [3], a game theory method for computing offloading is presented. Chen et al. [4] study the offloading problem using game theory and prove the existence of Nash equilibrium. The game theory-based or auction-based approaches lead to same disadvantage: they lack of the adaptability in task scheduling when the edge computing environment change dramatically [5].

Recently, researchers have proposed various task scheduling schemes based on deep reinforcement learning (DRL) method [6]. In [7], a DRL method is proposed for the multi-agent random game to reduce the energy consumption in MEC network, which is based on Nash equilibrium. A three-layer architecture is presented based on DRL to reduce the energy consumption in [8]. Also, the deep deterministic policy gradient (DDPG) algorithm [9] can be found in resource allocation problem [10]. However, the dimension of action space increasing exponentially will result in slow convergence in DRL schemes. Moreover, it is challenging to achieve an optimal tradeoff between exploration and exploitation in dynamic large-scale systems. The agent's behavior becomes too deterministic with the policy improving, resulting in lack of exploration to learn new actions. To tackle the curse of dimensionality and stochasticity in high-dimensional continuous action spaces, it is crucial in dynamic MEC environment.

Multi-access edge computing (MEC) has emerged as a promising framework for time-sensitive IoT applications, but limited edge server capacity, dynamic task arrivals, and high-dimensional action spaces are challenges for efficient task scheduling. Traditional methods like game theory lack adaptability, while DRL-based approaches suffer from slow convergence. The task scheduling issue in wireless edge computing is examined in this paper. Aiming to minimize energy consumption and the average service time for processing task, we propose an improved DRL-based task scheduling scheme in a software defined network (SDN) enabled MEC, where a centralized SDN controller is deployed on an edge server or cloud to make the task scheduling and resource allocation decisions [1].

The contributions of this paper are as follows:

(1) We formulate a comprehensive Markov decision process (MDP) model capturing MEC dynamics, enabling adaptive scheduling. We consider all kinds of task information, server status in MEC environment, construct the system state to capture the dynamic complexity of the MEC system, and then formulate a Markov decision process for the optimization problem. Different from the previous research work, as of the state representation of MEC environment, task characteristics and current workload are extracted as input features for deep reinforcement learning approach.
(2) As of DRL agent, we introduce ordinal optimization into the search process of DDPG-based approach to address slow convergence in the current DRL-based methods. In order to improve the learning efficiency, we combine ordinal optimiza-

tion with deep reinforcement learning for the first time to make efficient tradeoff between exploitation and exploration and achieve fast convergence in high-dimensional action spaces.

(3) We validate the approach in a simulation platform for task scheduling in dynamically changing MEC, in which we implement heterogeneous edge servers and fluctuation of tasks. Simulation results show 15–20% reductions in latency and energy consumption, with robust adaptability to workload spikes and server failures. It validates that the proposed learning approach can converge quickly and performs better than existing approaches in terms of convergence property, total service time as well as energy efficiency.

The rest of the paper is organized as follows: In Section 2, we firstly summarize related works and motivation, and then detail the improved learning algorithm for DRL scheduling methods. We present expriment results and discussion in Section 3. Finally, Section 4 concludes this article.

2 Methods

2.1 Related works and motivation

It is vey important to adaptively schedule tasks and computing power in the wireless edge computing. Many research aspects in cloud computing and edge computing, including task offloading and scheduling, computing services caching, have been studied [11–13]. In recent studies, DRL-based methods have been widely used to address computation offloading problems in MEC environment. Li et al. [14] use a combination of deep reinforcement learning (DRL) and an optimization-based approach to design task offloading rules, in which the long-term resource reservation is determined using DRL-based approach. In [15], Ju et al. propose a DRL-based approach to jointly solve resource allocation and offloading problem using a double deep Q-learning algorithm. Qiu et al. [16] study computation-intensive tasks offloading for blockchain-empowered mobile edge computing using DRL-based online method to adapt to MEC environments.

Considering dynamic task scheduling among multiple edge servers, Wang et al. adopt the software defined network (SDN) framework [17] and propose an adaptive resource allocation scheme based on deep reinforcement earning. In order to reduce energy consumption and delay at the same time, Ale et al. [18] study computation offloading among multiple edge servers and propose a DQN approach. In a distributed MEC system, load balancing has a direct impact on system performance. Liu et al. [19] focus on load balancing issues in edge computing and propose a network control approach using dueling double deep Q-learning network (D3QN) and LSTM. Xie et al. [20] consider task scheduling and load balancing jointly, propose a twin delayed deep deterministic policy gradient (TD3) algorithm to deal with high dimensionality of system states. Tang et al. [21] study serverless edge computing networks and propose a task scheduling algorithm using the dueling double deep recurrent Q-network (D3RQN) to solve partially observable stochastic game (POSG) problem. Xie et al. [22] study the serverless function deployment and model using directed acyclic graph. The task scheduling problem is optimized based on a dueling double deep Q-network. Recent studies further advance DRL-based MEC scheduling. A DRL approach is proposed for age of information aware

VNF scheduling in industrial IoT [23], an AI-based resource allocation scheme is developed for NFV-enabled aerial networks in [24], and a robust DRL framework is introduced for radio and computing management in vehicular networks [25].

The related works have illustrated the feasibility and advantage of deep reinforcement learning approach in designing the optimal decision-making policy and obtain higher utility. However, when applying the deep reinforcement learning methods in large state and action space, they suffer from long training time and slow convergence speed. In general, we hope to have pre-trained neural networks or other heuristic algorithms to obtain better training results for task scheduling in MEC network.

Ordinal optimization (OO) is a soft-computing tool for solving simulation-based optimization issues. The basic idea of OO has the following two points. Firstly, it is cheaper to obtain order than value in terms of computation cost, especially under conditions with prediction error and noise. Secondly, it is usually very difficult and time-consuming to find the best solution, instead easier to find the good enough solution. As opposed to directly solving a complex optimization problem, ordinal optimization is designed to improving the system performance by searching for good enough solutions through simulations. OO has been presented as a systematic soft-computing tool [26], and many researchers applied ordinal optimization to power systems, manufacturing system, and plug-in electric vehicles charging stations [27]. Inspired by the idea of ordinal optimization, we introduce ordinal optimization into the search process of deep reinforcement learning to effectively tradeoff between exploitation and exploration, and speed up the convergence to achieve high computation efficiency. Prioritized experience replay-DDPG prioritizes experience replay, improving sample efficiency but not exploration efficiency. Twin delayed DDPG, or bootstrapped DDPG enhances training stability with twin critics but increases computational complexity. Different from these DDPG variants, the proposed approach integrates an OO-enhanced exploration strategy to address high-dimensional action spaces. Simulations showed our approach outperform DDPG and TD3 with lower computation. Unlike prior OO-RL integrations, which apply OO to discrete Q-learning, our method integrates OO into DDPG's actor-critic framework and proposes binary-coded perturbations for action spaces in task scheduling for MEC.

2.2 Method description

In this section, we formulate the task scheduling in wireless edge computing as a discrete time Markov decision process (MDP) to capture the dynamic complexity of the edge computing system and present the DRL method based on objective function of task scheduling problem.

We study a MEC network in this paper, which consists of multiple IoT devices, several edge severs, and a remote cloud. These IoT devices connect to the MEC network mainly through wireless access point (AP), and edge servers are connected to access points by routers. The architectural diagram of the MEC system is shown in Fig. 1, and the parameters and variables in this paper are listed in Table 1. There are three layers in the above system's network topology: (1) cloud; (2) edge; and (3) device. IoT users generate task requests, which will be sent to edge servers or cloud server for process. Tasks generated by IoT users will first be assigned to their nearest AP nodes (entry nodes). If the entry node cannot process the arrived task with limited resource capacity in edge

Table 1 Notation definitions

Variables	Definition
G	the network topology
V	the set of access points
v_i	The ith AP node
C_i	Computing resource of MEC server v_i
$f_{r,j}$	Recommended frequency in server v_j
Q_i	Tasks queue in server v_i
$\omega_{i,j}$	Transmission rate of link
$\rho_{i,j}$	The shortest path between v_i and v_j
$\tau_{i,j}^{T}$	Task transmission time
$\tau_r{}^{C}$	Task computing time
τ_j^{R}	Current running task's remain time
$\tau_j{}^{W}$	Task waiting time
γ_r	Data size of request
ξ_r	Demanded CPU cycles of request
L_r	Deadline of request

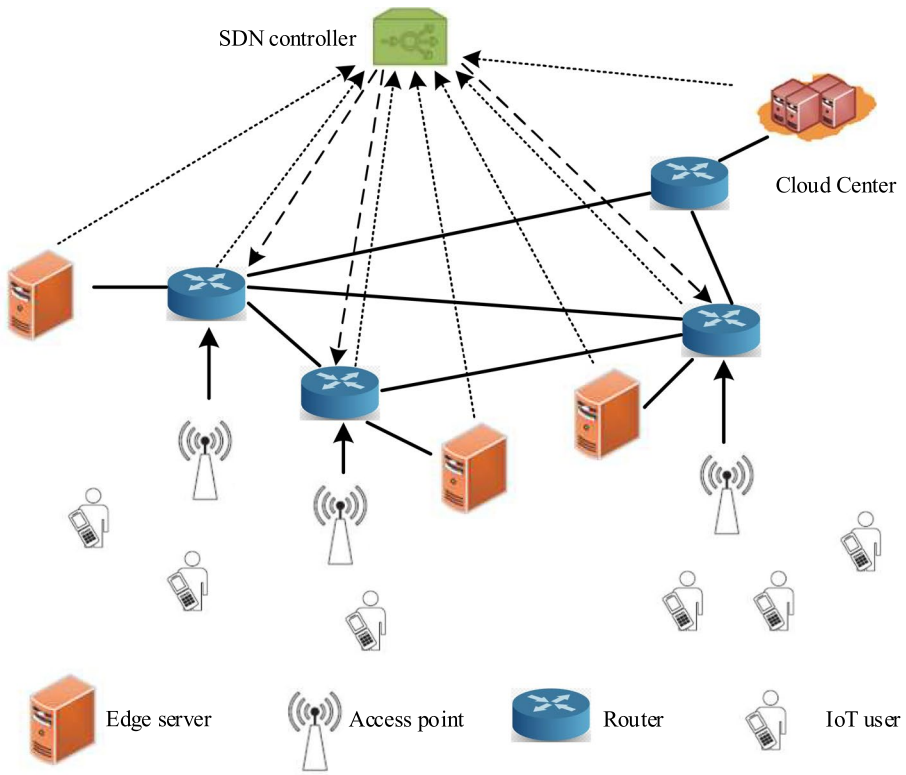


Fig. 1 The architecture diagram of MEC-based IoT networks. This figure presents the architecture of MEC-based IoT networks. It consists of a remote cloud center, several edge servers, routers, and many IoT users. IoT users connect to the MEC through access points. Tasks generated by IoT users will first be assigned to their nearest edge servers. There also exists a centralized SDN controller as coordinator, which can communicate with MEC servers periodically for servers state collection and task scheduling

servers, we need the scheduler to offload tasks to nearby cooperative edge servers. We adopt the software defined network (SDN) technology in the heterogeneous MEC architecture, which is illustrated in Figure 1. The advantage of SDN is that the data plane and control plane are separated from each other. The function of data plane includes data processing, forwarding, and status collection, etc., and the function of control plane is management of information on the data plane. Thus, the control plane plays the logical centralized control role in distributed networks. In our MEC architecture, there exists a centralized SDN controller as coordinator, which can communicate with the MEC servers periodically for servers state collection and task scheduling actions. The SDN controller collects state of all computing resources in edge computing system and arrived tasks information. We can deploy the SDN controller on any edge server or cloud server, and send scheduling actions to each edge server using the control plane function of software defined network.

We use an undirected graph $G = (V, E)$ to represent the network topology. V is the set of AP nodes in MEC network, represented by $V = \{v_0, v_1, v_2, \cdots, v_i, \cdots\}$. We utilize v_0 and v_i to denote cloud server and the i th AP node, respectively. E stands for the data links between AP nodes, that is, $E = \left\{e_{1,2}, e_{1,3}, \cdots, e_{i,j}, \cdots\right\}$. The shortest path between node v_i and node v_j is expressed by $\rho_{i,j}$.

Task execution process: A IoT user sends user data to the nearest entry node v_i, after receiving scheduling decision v_i offloads the task to the edge server with AP node v_j with the recommended CPU frequency $f_{r,j}$. The server executes the task and returns computation results. Figure 2 presents the workflow for task scheduling in a time slot, which is described in detail below.

The request-to-completion (RTC) process is defined as the period between end users generating a task and receiving results. For a specific task r, its metadata can be given as (γ_r, ξ_r, L_r), where L_r, ξ_r and γ_r are the deadline of task, the required CPU cycles and the data size. The whole RTC process includes three kinds of delay, i.e.,

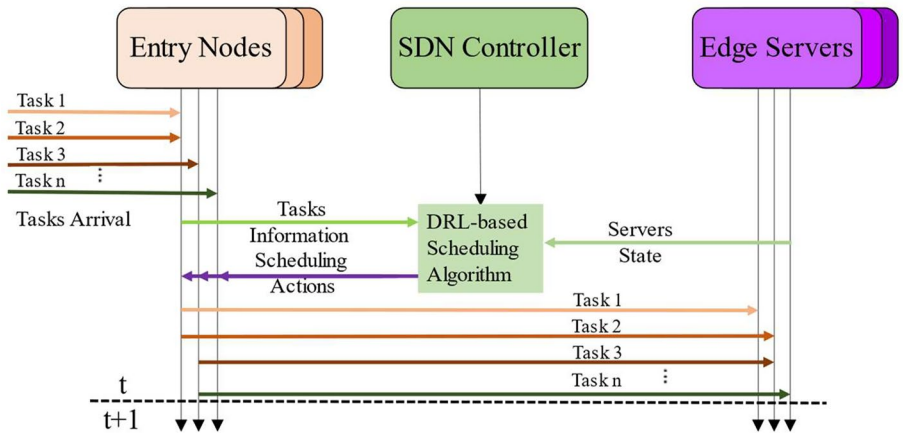


Fig. 2 The workflow for task scheduling in MEC. This figure presents the workflow for task scheduling in MEC. It consists of entry nodes, edge servers, and SDN controller. DRL-based scheduling algorithm runs on SDN controller. At time slot t, IoT tasks arrive at entry nodes, and tasks information is sent to scheduling algorithm as its input. The servers state is another input of scheduling algorithm. The scheduling actions generated by scheduling algorithm are sent to entry nodes. Then, scheduled tasks are offloaded to chosen edge servers

the access delay, the routing delay, and the computing delay. The access delay is the time spent on offloading the tasks to their nearest AP node, the routing delay is the time spent on scheduling tasks to another nodes, and the computing delay is the time spent on computing tasks in the selected edge or cloud server. We only consider the routing delay and the computing delay in this paper. Therefore, the total service time for request r includes the task transmission time $\tau_{i,j}^{T}$, the current running tasks remain time on target server τ_{j}^{R}, the computing time τ_{r}^{C} and the waiting time in the queue τ_{j}^{W}. The remaining time τ_{j}^{R} equal to current running task's computing time τ_{cr}^{C} minus its executing time τ_{run}, namely,

$$\tau_{j}^{R} = \tau_{cr}^{C} - \tau_{run} \tag{1}$$

The task queueing time in the queue, denoted as τ_{j}^{W}, is total computation time for all tasks arrived earlier at the task queue. We can calculate a task's computation time by dividing the number of CPU cycles needed by the computing frequency. The queueing time can therefore be determined by

$$\tau_{j}^{W} = \sum_{r=0}^{M} \frac{\xi_{*}}{f_{*,j}} \tag{2}$$

We can compute task transmission time $\tau_{i,j}^{T}$ by

$$\tau_{i,j}^{T} = \frac{\gamma_{r}}{\omega_{i,j}} \tag{3}$$

where $\omega_{i,j}$ is the transmission rate and γ_{r} is the data size of the task. One way to compute the transmission rate $\omega_{i,j}$ is by

$$\omega_{i,j} = \omega_{i,j}^{B} log_{2}\left(1 + \frac{P_{i,j} h_{i,j} L_{i,j}}{N_{0}}\right) \tag{4}$$

where $\omega_{i,j}^{B}$ is the bandwidth, $P_{i,j}$ is the transmission power, $h_{i,j}$ and $L_{i,j}$ are Rayleigh fading and path loss, respectively.

The computing time τ_{r}^{C} can be calculated by

$$\tau_{r}^{C} = \frac{\xi_{r}}{f_{r,j}} \tag{5}$$

where $f_{r,j}$ and ξ_{r} are the allocated computing frequency and the required CPU cycles of task, respectively.

If an arrived task can be processed on the edge server, we denote the total service time for task as

$$T_{r} = \tau_{i,j}^{T} + \tau_{j}^{R} + \tau_{j}^{W} + \tau_{r}^{C} \tag{6}$$

If we consider all arrived tasks in MEC network, the average completion time (ACT), can be computed by

$$\mathrm{ACT} = \frac{1}{N_t}\sum_{r=1}^{N_t} T_r \tag{7}$$

where N_t is the total number of tasks in MEC systems arrived at time slot t.

In this paper, we denote the energy consumption for transmission by E_r^T and energy consumption for computation by E_r^C. Then, the total energy consumption E_r is computed as follows.

$$E_r = E_r^T + E_r^C \tag{8}$$

The transmission energy consumption can be computed as follows.

$$E_r^T = \tau_{i,j}^T P_{i,j} = \frac{\gamma_r}{\omega_{i,j}} P_{i,j} \tag{9}$$

One way to compute the computation energy consumption is

$$E_r^C = c\left(f_{r,j}\right)^2 \xi_r \tag{10}$$

where $c = 10^{-26}$ and $f_{r,j}$ is the task computing frequency allocated for request r on server v_j [18].

To improve the QoS for IoT users, we try to minimize the total service time of all tasks. In this paper, our objective is to reduce total service time and energy consumption of the MEC system. Thus, we can describe the task scheduling in MEC system as an optimization problem, that is,

$$\min \sum_{r=1}^{N_t} (\mu_1 T_r + \mu_2 E_r) \tag{11}$$

where $\mu_1$and μ_2 are weighted factors. These coefficients can be changed in different applications.

For applying DRL method to task scheduling, we formulate the above optimization problem as a discrete time MDP, and an MDP model is usually denoted as a tuple $(S, A, P, R(\cdot), T)$. Here, S,A,P,$R(\cdot)$ stand for the state space, the action space, the transition probability, and the immediate reward function, respectively. T is the horizon of time. In DRL scheduling method, through the continuous interaction with the MEC environment, the DRL agent can take actions and obtain corresponding rewards and aims to maximize cumulative rewards over time, as shown in Fig. 3.

Data preprocessing and features extraction is very critical in this method. The MEC environment includes factors such as network conditions, tasks information, available computing resources, and current workload. These factors are represented as input features to the DRL agent. These features, including servers state, and tasks information, are time-varying. In this work, we capture the system dynamics of the MEC environment by selecting the extracted tasks characteristics, and the accessible computing resources and current workload of all servers as system states. The DRL formulation is detailed as follows.

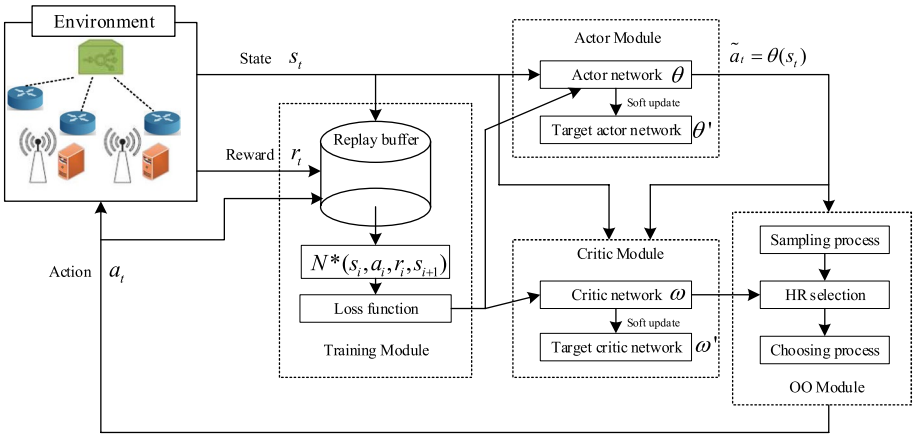


Fig. 3 The detailed framework of proposed learning algorithm. This figure presents the schematics of the proposed DRL algorithm. There are two main parts: MEC environment and DRL model. The DRL model consists of four modules, i.e., Actor module, Critic module, Training module, and Ordinal optimization module. The actor network, target actor network, critic network, and target critic network are trained by history data restored in Replay buffer, which is in the format of transition tuples (state, action, reward, and next state). The training module randomly samples a minibatch of transition tuples and computes the loss function. After an action is output by the actor network, it goes through the OO module to get one good enough action with high probability in its neighborhood. The latter is chosen as the action to the environment

(1) State: At time slot t, system state s_t consists of extracted tasks characteristics and the edge servers state, which is denoted as $s_t = (\mathcal{Q}(t), \mathcal{E}(t))$. The arrived tasks on all entry nodes in edge system is represented by

$$\mathcal{B}(\mathrm{t}) = \left[\tau_1^1, \cdots, \tau_i^k, \cdots, \tau_N^1, \cdots\right] \tag{12}$$

For each task τ_i^k, it is expressed by the different task metadata (γ_r, ξ_r, L_r), where L_r, ξ_r and γ_r are the deadline of task, the required CPU cycles and the data size. $\mathcal{Q}(t)$ is extracted characteristics of tasks to be scheduled, which can be expressed as

$$\mathcal{Q}(t) = \left[n_{e_1}, \cdots, n_{e_i}, \cdots, n_{e_N}\right] \tag{13}$$

The dynamics of MEC severs can be modeled by

$$\mathcal{E}(t) = \left[\mathcal{E}_{e_1}, \cdots, \mathcal{E}_{e_i}, \cdots, \mathcal{E}_{e_N}\right] \tag{14}$$

where $\mathcal{E}_{e_i} = [Q_i, C_i]$ denotes the pending tasks queue and available CPU resource of the server with AP node v_i.

(2) Action: For a specific task τ_i^kin the edge system, it can be processed at the edge server directly connected to the entry node v_i or scheduled to another edge server v_j for better performance. Each task can only be scheduled to one virtual machine (VM) on edge server for process. For edge servers, VMs are denoted as different levels according to various processing frequencies. Therefore, we divide virtual machines on edge server for processing tasks into L levels. For example, we may consider five levels, according to CPU frequencies such as 250 GHz,200 GHz, 150 GHz, 100 GHz, and 50 GHz. . Based on the system state s_t at time slot t, the action of the DRL agent is defined as

$$a_t = \left[A_1^1, \cdots, A_i^k, \cdots, A_N^1, \cdots\right] \tag{15}$$

where A_i^k is scheduling actions for tasks τ_i^k, and we denote it as

$$A_i^k = [j, l] \tag{16}$$

Where j represents the index of selected edge server, l represents the recommended VM level. We adopt binary strings to express the action variables.

(3) Reward: After taking a possible action a_t, the DRL agent can acquire a certain reward, which needs to reflect the objective of the system. Considering the optimization index in Eq.(11), we adopt the following reward of the MEC environment.

$$\mathcal{R} = \sum_{r=1}^{N_t} \mu_1 (L_r - T_r) - \mu_2 E_r \tag{17}$$

When the task processing delay exceeds the deadline, a negative reward item is returned.

For a current state, the state-action value function (Q-function) is used to evaluate the long-term reward for all possible actions. Therefore, we use the following Q-function.

$$Q^{\pi}(s, a) = \mathbb{E}_{\pi}\left[\sum_{l=0}^{\infty} \gamma^l \mathcal{R}_{t+l} | s_t = s, a_t = a\right] \tag{18}$$

The discount factor $\gamma \in [0, 1]$ is used to tradeoff between future reward and current reward. Thus, our objective is to learn an optimal policy π^*. Under arbitrary initial conditions, it can output action to obtain the largest Q-function, which can be defined as

$$\pi^* = \text{argmax} Q^{\pi}(s, a) \tag{19}$$

The proposed design method is compatible with SDN protocols (e.g., OpenFlow). The SDN controller uses standard APIs to collect server states and deliver scheduling actions, while the DRL agent operates as a MEC application, compatible with existing 5 G and IoT standards.

The proposed methods aim to optimize scheduling and satisfy MEC constraints: task deadlines $T_r \leq L_r$, server capacity $\sum f_{r,j} \leq C_j$, and bandwidth limits $\gamma_r / \omega_{i,j} \leq \rho_{i,j}$. The DRL agent selects actions $A_i^k = [j, l]$ ensuring resource allocation within these bounds.

2.3 Improved learnig algorithm for DRL scheduling method

When scheduling numerous computing tasks to multiple edge servers in dynamically changing MEC environment, an increase in the number of IoT devices, tasks, or edge servers would bring an exponential growth of the action space. In these situations, it is particularly difficult to ensure that the DRL method can converge to a stable point. In traditional deep reinforcement learning such as deep Q-learning (DQN), a neural network is used to learn the long-term rewards or probabilities for any state-action pairs under system state s_t. When applied to issues with high-dimensional action spaces, the convergence is very slow since it needs a thorough search through all actions. To address these problems, DDPG- or TD3-based approaches utilize both policy-based algorithms with value-based algorithms.They are better suited for stochastic

environments or situations with high-dimensional or continuous actions because they can express continuous actions. An action-value function and a policy function are both used by DDPG. There are actor neural network and critic neural network in the actor-critic architecture. The former is used to optimize directly the policy function; and the latter is utilized to estimate the action-value function. In other words, the actor network generates actions, while the critic network assesses those actions. However, the convergence and accuracy of the neural networks are the main difficulties in applying DRL to solve task scheduling problems. These methods such as DDPG can can search action spaces with high dimension, but they also suffer from slow convergence. In order to search the huge action space, exploration methods are needed. For example, traditional exploration methods such as the ϵ-greedy, which is used in DQN, are not efficient. In maximum entropy reinforcement learning methods such as soft actor-critic (SAC), adding entropy bonus is a way to achieve sufficient exploration. In order to fully explore all actions, it is time-consuming and leads to low computation efficiency.

The schematics of the proposed DRL scheduling method is illustrated in Fig. 3. The DRL model is the core part of the proposed method. It consists four modules, which are detailed below.

The actor module consists of a DNN. We denote the parameter of the DNN as θ, which is randomly initialized. It works as a policy function which outputs a feasible action $\tilde{a}_t$ at time slot t, taking the state s_t as the input. The input–output relation is expressed as

$$\tilde{a}_t = \theta(s_t) \tag{20}$$

We denote a copy of the actor networks as target actor network with parameters θ', which are used for calculating the target values.

The critic module consists of a DNN. We denote the parameter of the DNN as ω, which is randomly initialized. We use critic network to compute the reward of a state-action pair $(s_t, \tilde{a}_t)$, which can be defined as a mapping

$$q_t = \omega(s_t, \tilde{a}_t) \tag{21}$$

We also denote a copy of the critic networks as target critic network with parameters ω'.

The ordinal optimization module plays a very improt role in our DRL algorithm. We use actor network to predict the preliminary scheduling action and adopt OO to find the optimal action within a small distance to the actor network's output when it gradually reaching the optimal policy over time. As shown in Fig. 4, the decision space Θ is a set of all possible candidate solution θ, where $\theta \in \Theta$. The top-g solutions are named as the "good enough" subset G, where g is the size of the subset G. The subset S is a select subset, where s denotes the size of the subset S. It is the selection rule of subset S that has an important effect on the probability of finding the optimal solution. The probability of finding the optimum solution under a noise is denoted by alignment probability (AP) $P(|G \cap S| \geq k)$. k is the alignment level, that is, number of solutions which are both members of subset S and G. Furthermore, we can increase the size of subset S and G to make alignment probability more accurate.

According to the "No-free-Lunch" theory, the essence of OO is that it provides a way to tradeoff between computation burden and system performance. Instead of

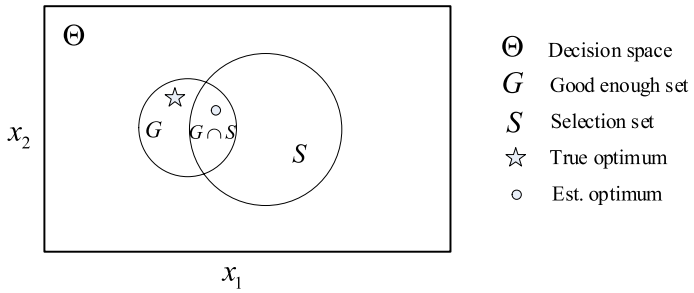


Fig. 4 The concept illustration of ordinal optimization. This figure illustrates the basic idea of ordinal optimization (OO). There are two sets in the decision space. G denotes good enough set, and S stands for a selection set. The true optimum lies in good enough set. The estimated optimum lies in the intersection of these two sets. OO focus on how to get one good enough decision with high probability

finding the time-consuming optimal solution, it is better to find the "good enough" solution with less computation burden. The advantage of OO is to evaluate solutions and differ good solutions from bad solutions using coarse models, which does not affect the order of solutions even with some errors.

As shown in Fig. 3, the OO module consists of the following three components. (1) Sampling process: Generate N_a actions in decision space to enrich the set of candidate actions by imposing random perturbation in bits binary-coded solution to the output of the actor network. (2) HR Selection: Top-s actions are picked from the N_a actions using the critic network, which consists of the select subset. The main purpose of OO is to find a subset, which have good enough solutions with high probabilities, by using a coarse model with less computation burden, which reduce significantly the time spent in evaluating solutions. Selection rules play an important role in the successful application of OO, and they include horse racing (HR), blind picking (BP), and round robin (RR). In this paper, horse racing with global comparison is used with ordinal optimization to narrow down the search space for the optimal scheduling action. (3) Choosing process: Evaluate all actions in the select subset using objective function to select the optimal action.

Ordinal optimization (OO) enhances exploration efficiency in high-dimensional discrete action spaces by reducing the effective search space through coarse evaluations.Let π denote the action space with $|\pi|$ possible actions.Traditional exploration methods like ϵ-greedy require evaluating a large fraction of π, leading to a computational complexity of $O(|\pi|)$. In contrast, OO samples a subset of $N_a \ll |\pi|$ candidate actions and selects the top-s actions based on a coarse critic evaluation, reducing the complexity to $O(N_a + s)$. With the alignment probability $\mathrm{P}(|G \cap S| \geq k) = 0.95$, OO ensures a high likelihood of including "good enough" actions in the selected subset S, reducing the search space complexity compared to traditional exploration methods like ϵ-greedy or entropy-based approaches.

The training module consists of three procedures: data preprocess, training data generation, and data-driven learning. The following is the method used during the training procedure. As shown in Figure 3 and Algorithm 1, we first initialize the experience replay buffer $\mathcal{B}$ with the size of B as well as the actor and critic network with random parameters θ and ω. The episode is the task scheduling process in MEC

that schedule tasks to target server. An initialize state is obtained in each episode. At time slot t, state s_t is the input of the actor network, and the OO Module outputs final action a_t. After executing action a_t, the reward r_t and the next state s_{t+1} can be obtained from MEC environment. We store the transition tuple (s_t, a_t, r_t, s_{t+1}) in replay buffer to update the actor and critic network.

If the data are given into the model sequentially, there may be a significant degree of correlation between the sequence of transitions during the agent's interactions with the MEC environment. To prevent oscillating or diverging in trainning, we first randomly sample a minibatch of transition tuples $(s_i, a_i, r_i, s_{i+1}), i = 1, \cdots, N$ from replay buffer in network training process. This is helpful to break the correlation between samples and improve data utilization.

Then, the parameters of critic network can be updated to minimize the loss function, which is the mean square error (MSE) between the outputs of critic network and its target values,

$$L(\omega) = \frac{1}{N}\sum_{i=1}^{N}\left(\hat{q}_i - \omega(s_i, a_i)\right)^2 \tag{22}$$

where $\hat{q}_i = r_i + \gamma\omega'(s_{i+1}, a_{i+1})$, and $a_{i+1} = \theta'(s_{i+1})$, γ is the discount factor.

For the actor network, the parameters are updated in the direction of minimizing the loss function, which is defined by

$$L(\theta) = -\frac{1}{N}\sum_{i=1}^{N}\Big[a_i^T \log(\theta(s_i)) + (1 - a_i)^T \log(1 - \theta(s_i))\Big], \tag{23}$$

where $\theta(s_i)$ denotes the output of the current actor network with parameters θ and input s_i. The loss function is the averaged cross-entropy loss of the selected samples.

Algorithm 1 Improved task scheduling Algorithm

Input: The arrived tasks in MEC system and the edge servers state.
Output: The scheduling actions for all tasks: target server index, recommended VM level.

Randomly initialize the actor network and critic network with parameters θ and ω.
Initialize the target actor network and target critic network with parameters $\theta' \leftarrow \theta$ and $\omega' \leftarrow \omega$.
Given the size B of the replay buffer $\mathcal{B}$, initialize the replay buffer.
Given control parameters.
for $episode = 1$, M **do**
Observe an initial state s_1
for $t = 1$, T **do**
At time slot t, output action a_t using OO Sampling and Choosing procedure for input state s_t.
Execute action a_t, observe reward r_t and new state s_{t+1}
Store transition tuple (s_t, a_t, r_t, s_{t+1}) in replay buffer.
Sample a random minibatch of N transitions (s_i, a_i, r_i, s_{i+1}) from replay buffer and update θ and ω as Eq.(22) and (23).
if t **mod** C **then**
update the target networks as Eq.(24) and (25).
end if
end for
end for

In order to prevent divergence and oscillation during training, the target actor and critic network keep their parameters unchanged in a certain period. Specifically, we

use the data in the replay buffer to update actor and critic network every time slot, and softly update the target networks every C time slots.

$$\theta' \leftarrow \tau\theta + (1-\tau)\theta' \tag{24}$$

$$\omega' \leftarrow \tau\omega + (1-\tau)\omega' \tag{25}$$

Here, τ is the soft update coeffient, and C is a control factor, that is, frequency to update the target networks. A study was conducted with different C value. Smaller C (e.g., 1) may cause oscillations, while larger C (e.g., 5) led to slow convergence. These results suggest that $C = 3$ is suitable for the MEC environment.

Algorithm 2 OO sampling and choosing procedure

1: Generate N_a actions by imposing random perturbation in bits binary-coded solution to the input action.
2: Select top-s actions from the N_a actions using the critic network.
3: Choose the optimal action in the select subset using objective function.

The proposed method is shown in Algorithm 1 and Fig. 3, and it operates as the following flow chart:

(1) At time slot t, the SDN controller collects state $s_t = (\mathcal{Q}(t), \mathcal{E}(t))$.
(2) The actor network outputs a preliminary action $\tilde{a}_t$.
(3) The OO module generates $N_a = 100$ candidate actions via binary perturbations.
(4) The critic network selects the top-$s = 10$ actions.
(5) The optimal action is chosen based on the objective function (Equation (11)).
(6) The action is executed, and the transition is stored in the replay buffer for training.
(7) Training actor network and critic network.
(8) $t = t$+1, Goto 1).

The proposed algorithm's convergence is supported by DDPG's theoretical guarantees under Lipschitz-continuous rewards and bounded gradients [9]. In OO sampling and choosing procedure, we select top-s actions from the N_a actions by using the critic network to evaluate state–action pairs.The prerequisite for using the critic network in the OO procedure is the convergence of the Q-function. The critic network of the proposed algorithm converges to the optimal value function (Proof is included in Appendix). By using alignment probability $P(|G \cap S| \geq k) \geq 0.95$ in OO, it ensures the selected action is near-optimal with high probability, accelerating convergence.

The proposed algorithm achieves local optimality due to OO seeking "good enough" solutions within a sampled subset. Given the alignment probability above, the selected action is within the top 5% of the action space with high probability. While global optimality is not guaranteed due to the NP-hard nature of task scheduling, the DRL framework maximizes long-term rewards, and it approximates global optimality over time.

The binary encoding scheme represents each action $A_i^k = [j, l]$ (server index j and VM level l) as a binary string, ensuring full coverage of the discrete action space. For N servers and L VM levels, the action space size is $N \times L$, and each action is uniquely encoded

with $\lceil \log_2(N) \rceil + \lceil \log_2(L) \rceil$ bits. Perturbations are applied by flipping bits to generate candidates within a Hamming distance, which can provide fine granularity for small action spaces. However, for very large action spaces, the fixed-length encoding may limit granularity, and potentially make some optimal actions less likely to be sampled. To mitigate this, we set $N_a = 100$ to ensure sufficient solution diversity.

The binary encoding scheme can handle the increasing numbers of servers or tasks, for instance, binary string with 5 bits corresponding to server index range of [0, 31]. But for larger action space, we may increase N_a to improve performance, therefore increasing computational overhead.

The proposed algorithm mitigates training instability through the following design factors. The experience replay buffer breaks temporal correlations to reduce divergence due to non-stationary target values. Soft updates stabilize target network updates to mitigate overestimation bias in the critic. Additionally, OO's coarse evaluations in the sampling process reduce sensitivity to critic overestimation by prioritizing "good enough" actions, which can enhance robustness in dynamic MEC environments.

3 Results and discussion

3.1 Experiments description

We adopt Python/Pytorch as the programming platform to simulate SDN-enabled MEC environment. We design a simulation platform for computing power and task scheduling in fluctuating dynamic MEC scenario, which is a testbed for applying and testing the proposed approach. Furthermore, we execute the simulation test under different parameters, discuss the test results, and compare the proposed approach with other methods. According to the schematics in Fig. 3, simulation platform includes two parts, i.e., the MEC environment and the DRL model. The environment is made of three components: the tasks arrival, server nodes, and network. Moreover, the MEC environment can update its state, such as pending tasks queue and computing resources in MEC. The edge servers also receive scheduling actions from DRL model, execute the tasks, and return the utility. The utility is computed according to the service time and energy consumption of all tasks. Specifically, an edge server executes a scheduled task with the allocated frequency, and then, the utility can be obtained with required CPU cycles and the computing frequency. As described in previous section, the DRL model consists four modules: Actor module, critic module, OO module, and training module. The DRL model learns the policy function from interaction with the environment and outputs scheduling actions. In this simulation, the neural networks in DRL consists of three layers, which are fully connected layers, and the number of the neurons in two hidden layers are both 256. The simulation parameters are given in Table 2.

In the MEC network, there are four servers with the same computing resources, 100 Intel i7-1195G7 CPU modules with maximum 500GHz processing capability. To reflect the unbalanced traffic property of the practical network, the numbers of tasks arriving from mobile devices to each AP node are randomly set between 3 and 6 in each time slot. The required CPU cycles is set between 8 Megacycles and 2 Gigacycles, and the data size of tasks offloaded by mobile devices is set between 200 KB to 20 MB. The data transmission rate is set between 250 and 1,000 Mbps.

Table 2 Evaluation parameters

Parameter	Value
Task data size (Bits)	$[2 \times 10^5, 2 \times 10^7]$
Task CPU cycles (cycles)	$[8 \times 10^6, 2 \times 10^9]$
Server max frequency (GHz)	500
Learning rate α	5×10^{-4}
Discount factor γ	0.99
Minibatch size	32
Soft update coefficient	0.005
Size of the subset **G**	10
Alignment probability	0.95
Alignment level	5

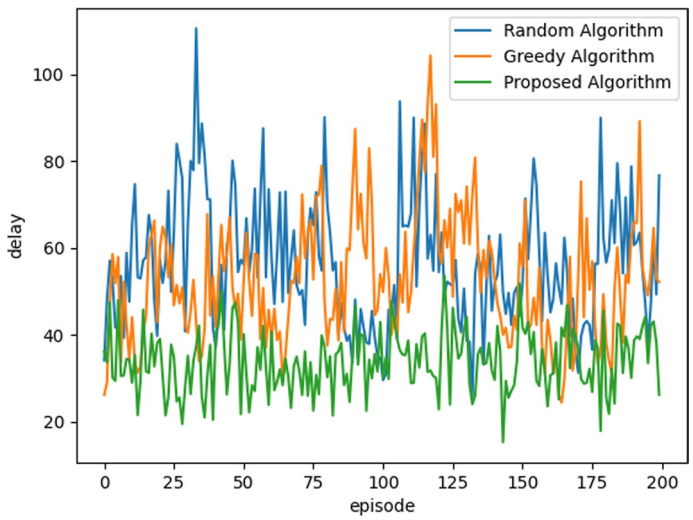


Fig. 5 Service time comparison. This figure shows the performance comparison of total service time among the proposed approach, random algorithm, and greedy algorithm. The proposed approach's delay is smaller than the other two

3.2 Experiments results

We compare the proposed algorithm with the following baseline algorithms in simulations. (1) Random Algorithm: This algorithm randomly generates one solution as the scheduling action. (2) Greedy Algorithm: This algorithm firstly generates multiply actions by random, and output the action to optimize the objective. (3) DDPG and TD3 as state-of-art DRL algorithms are also implemented to be our baseline algorithms. The metrics to evaluate the proposed algorithm and other baseline algorithms include rewards of each time slot, total service time and total energy consumption of tasks.

Firstly, we compare the proposed algorithm with the random algorithm and greedy algorithm according to total service time and system energy consumption. The results are given in Figs. 5 and 6. From these results, it can be seen that the improved learning approach significantly performs better than random algorithm and greedy algorithm. The greedy algorithm is used to output actions with best reward (for example, minimal delay or minimal energy consumption) according to the optimization objective of system

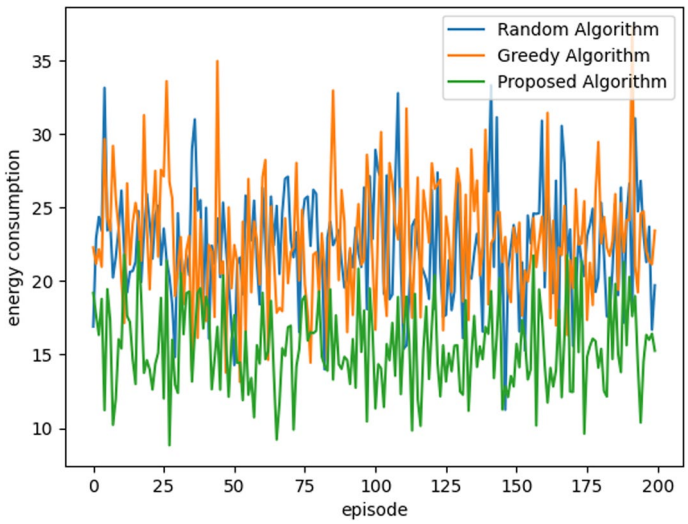


Fig. 6 Energy consumption comparison. This figure shows the performance comparison of system energy consumption among the proposed approach, random algorithm, and greedy algorithm. The proposed approach's energy consumption is smaller than the other two

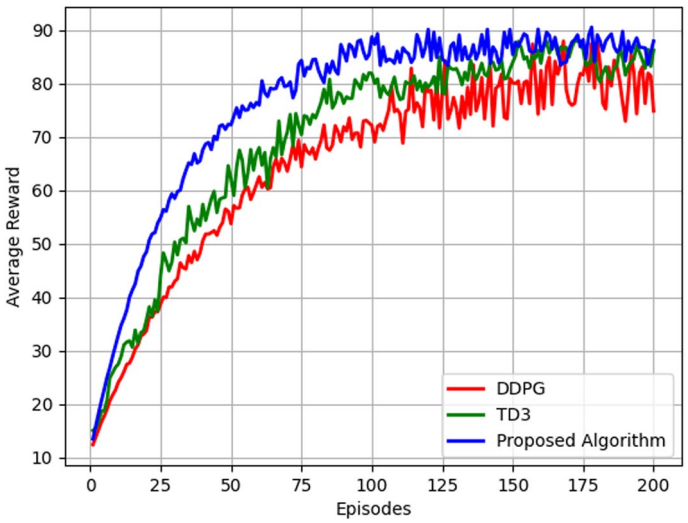


Fig. 7 Convergence comparison. This figure shows the comparison of average reward per episode among the proposed approach, DDPG and TD3. The proposed approach obtain higher rewards than others. The lower variance and higher reward indicate faster and more stable convergence. DDPG shows some fluctuations in the later training, while TD3 is more stable than DDPG, and obtains higher rewards

among a lot of randomly generated decision actions. Therefore, it can usually obtain more reward and better performance than the random algorithm. However, it fails to locate the optimum with rapidly increase of computation overhead when the decision space is huge.

Figure 7 shows the experiment results among the proposed approach and other two DRL-based approaches. For fair comparison, DDPG and TD3 used the same neural networks as the proposed algorithm and same hyperparameters: learning rate of 5×10^{-4}, discount factor of 0.99, minibatch size of 32, and soft update coefficient of 0.005. Hyperparameters were tuned via grid search over learning rates $[10^{-5}, 10^{-3}]$.

To compare the convergence, the average reward per episode for the proposed method, DDPG, and TD3 over 200 training episodes are shown in Fig. 7. In experiments, we computed the average reward per episode over 5 runs to show the statistical significance of improvements.

As shown in Fig. 7, the proposed method achieves an average reward of 85.6 ± 4.2 after 100 training episodes, compared to 76.3 ± 6.8 for DDPG and 81.1 ± 5.9 for TD3, with 95 % confidence intervals. The lower variance and higher reward indicate faster and more stable convergence. DDPG shows some fluctuations in the later training, while TD3 is more stable than DDPG, and obtains higher rewards. The proposed method outperforms TD3 and DDPG.

As mentioned before, the MEC environment is dynamically changing with uncertainty of tasks arrival and changing processing capacity of edge servers. The impact of total CPU cycles of tasks and processing capacity of the servers on service time are shown in Figs. 8 and 9, respectively.

3.3 Analyis and discussion

In experiments, the proposed algorithm can quickly converge because of high computation efficiency of ordinal optimization. The OO procedure explores better actions in a small neighborhood of the action output of the actor network, which can help the actor network converge to the optimal policies. That is, the actor neural network search for optimal actions, and the ordinal optimization is introduced to tradeoff between exploration and exploitation. Based on the binary-coded action output by the actor network, a candidate action set is constructed according to Hamming distance in its neighborhood. Then, all candidate actions are ordered using the critic network, and the top-s actions with high state-action value are selected as the selection set. This step is kind of similar to the DQN network, but the difference is that the DQN network only chooses the action with maximum state-action value. Finally, we

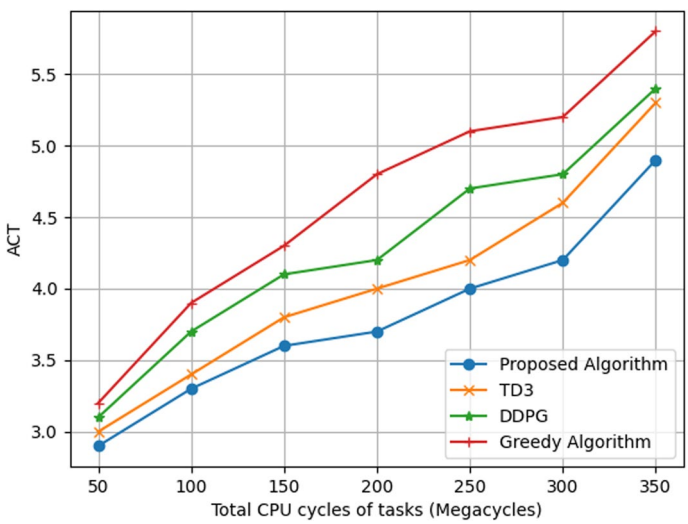


Fig. 8 The impact of total CPU cycles of tasks. This figure illustrates the impact of total CPU cycles of tasks on Average completion time (ACT). With the increase of total computation amount, the total delay increases, and the ACT increases. Our algorithm obtain the smallest service time among all algorithms

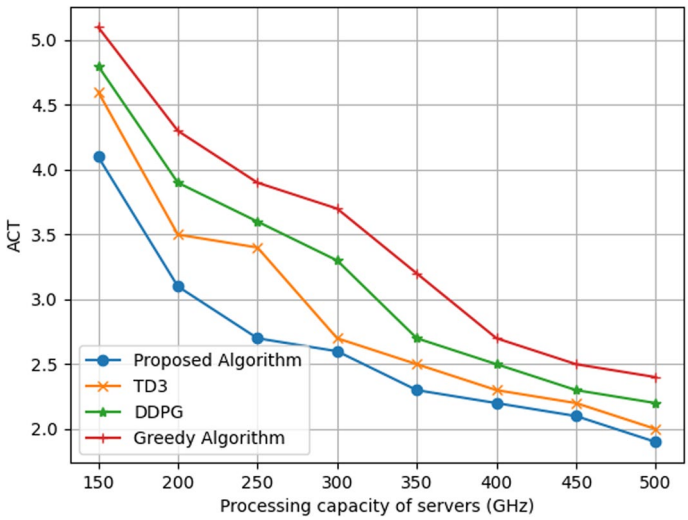


Fig. 9 The impact of processing capacity of servers. This figure illustrates the impact of processing capacity of servers on Average completion time (ACT). It shows that the computing time will decrease as the processing capacity of the servers increases. As a result, the ACT will also decrease. Our algorithm performs better than other algorithms

can obtain the best action according to the optimization objective among these candidate actions, the number of which is considerably small.

To validate adaptability of task scheduling algorithms in dynamically changing environments, we will analyze the impact of computing requirement of tasks and processing capacity of servers on ACT below. In Fig. 8, we can observe that, with the increase of total computation amount, the total delay increases, and the ACT increases. It can be seen from Fig. 9 that the computing time will decrease as the processing capacity of the servers increases. As a result, the ACT will also decrease. In these simulation results, our algorithm performs better than other algorithms.

The DRL agent updates adaptively state and utilizes OO's coarse evaluations, so that the algorithm is robust to information uncertainties, such as noisy task metadata or server state estimates. Simulations with 10% noise in γ_r and ξ_r showed a 3–5% increase in service time.

The proposed algorithm's computational complexity is $O(N_a + s)$ per time slot, where $N_a = 100$ actions are sampled and $s = 10$ are evaluated, compared to $O(|\Theta|)$ for exhaustive search. Signaling overhead includes task metadata transmission to the SDN controller (approximately 1 KB per task) and action deliver (100 bytes per action), totaling less than 10 KB per time slot for six tasks. This overhead is negligible for MEC applications with tight latency budgets. Therefore, it is feasible for MEC networks with 1 Gbps links.

The binary-coded solution and perturbation for generation is effective for the simulated MEC network with four servers and five VM levels. However, for larger action spaces (e.g., 10–20 servers) or complex environments with heterogeneous VMs or variable server types, it may increase computational overhead. Therefore, the number of sampled actions N_a need be increased, potentially impacting real-time performance. In future work, we will explore hierarchical action representations or adaptive sampling strategies.

For real-time application, the greedy method evaluates 100 randomly generated actions per step and selects the one with the best objective according to Equation (11), which need more computation effort compared to the proposed method. For OO, only top-s actions need evaluated using Equation (11), while N_a =100 actions are evaluated using trained critic network to select the top-s actions, which reduce search effort and is the advantage of OO. However, the greedy approach lacks guided exploration, leading to suboptimal performance. It can be noticed from this work that the exploration mechanism to improve the efficiency in search process is the key to solve the convergence problem of DRL scheduling methods, which could matter in harder tasks.

The proposed algorithm was tested with four homogeneous servers, but it can generalize to heterogeneous edge servers and non-uniform data rates. The state representation dynamically captures server-specific features, with adaptation to heterogeneous configurations. However, non-uniform data rates may increase transmission time variability, requiring larger N_a to maintain performance. Simulations with heterogeneous servers (e.g., mixed CPU types) showed a 5–10% increase in service time.

Under sudden spikes in workload or server failure scenarios, which correspond to the increase of total computation amount and the decease of servers capacity respectively in the previous subsection, the algorithm updates the state to DRL agent in real-time to adapt the dynamic environments. For server failures, the SDN controller routes tasks to available servers, with a 5–10% service time increase during single-server failure. These results show adaptability to dynamic changes.

4 Conclusion

In order to support latency-sensitive and computation-intensive IoT applications, we have studied the task scheduling for wireless edge compuution in this paper. We firstly formulated the task scheduling for SDN-enabled MEC network as a minimization problem of average service time of tasks and system energy consumption. To address above problem, we summarized it as a discrete time Markov decision process and proposed an improved DRL-based scheduling algorithm, which deploys a computation-efficient OO tool to optimally achieve the exploitation/exploration tradeoff in the DRL algorithm in order to ensure fast convergence. The experiment results validate that the proposed learning approach is better than existing approaches in terms of the convergence property, the total service time as well as the energy efficiency. Simulation results also show that the proposed approach can achieve adaptive task scheduling in dynamically changing MEC network.

Appendix

Theorem and Proof

Theorem

The critic network of the proposed algorithm converges to the optimal Q-function.

Proof

In OO sampling and choosing procedure, we select top-s actions from the N_a actions by using the critic network to estimate $Q(s_t, a_t)$ for each state-action pair. Considering a scheduling policy π, the system follows the state transition probability, which can be expressed as:

$$\Pr\{s_{t+1}|s_t, \pi(s_t)\} = \prod_{i=1}^{N} \Pr\{s_{t+1}^i|s_t^i, \pi(s_t^i)\}, \tag{26}$$

Because the state space $\mathbb{S}$ and action space $\mathbb{A}$ are finite, the state transition probabilities in Equation (26) are stationary, and all state-action pairs $\{(s_t, s_t)|t \in \mathbb{N}^+\}$ can be visited infinitely. Given (s_t, a_t, s_{t+1}, r_t), the update rule of the critic network follows [9]

$$Q(s_t, a_t) = Q(s_t, a_t) + \alpha(s_t, a_t) \times \left[r_t + \gamma \max_{a \in \mathbb{A}} Q(s_{t+1}, a) - Q(s_t, a_t)\right]. \tag{27}$$

Subtracting $Q^*(s_t, a_t)$ from both sides of equation (27), we define

$$\Delta(s_t, a_t) = Q(s_t, a_t) - Q^*(s_t, a_t), \tag{28}$$

and

$$F(s_t, a_t) = \left[r_t + \gamma \max_{a \in \mathbb{A}} Q(s_{t+1}, a)\right] - Q^*(s_t, a_t), \tag{29}$$

then we obtain

$$\Delta(s_t, a_t) = (1 - \alpha)\Delta(s_t, a_t) + \alpha(s_t, a_t)F(s_t, a_t) \tag{30}$$

For $F(s_t, a_t)$ in equation (30), we derive

$$||\mathbb{E}[F(s_t, a_t)|F]||_\infty = \Pr(s_{t+1}|s_t, a_t)F(s_t, a_t) \tag{31}$$

$$\leq \gamma ||Q(s_t, a_t) - Q^*(s_t, a_t)||_\infty \tag{32}$$

$$= \gamma ||\Delta(s_t, a_t)||_\infty. \tag{33}$$

and

$$\text{var}[F(s_t, a_t)|F] = \text{var}\left[r_t + \gamma \max_{a \in \mathbb{A}} Q(s_{t+1}, a)|F\right]. \tag{34}$$

Since r_t is bounded, we get

$$\text{var}[F(s_t, a_t)|F] \leq C(1 + ||\Delta(s_t, a_t)||_\infty^2) \tag{35}$$

Here, $C > 0$ is a constant. Because $\sum_{t=0}^{\infty} \alpha$ is infinite and $\sum_{t=0}^{\infty} \alpha^2$ is finite, $\Delta(s_t, a_t)$ converges to zero with probability 1 when equations (33) and (35) satisfy. Therefore, it means the critic network converges to the optimal Q-function $Q^*(s_t, a_t)$. □

Abbreviations

ACT	Average completion time
AP	Access point
DL	Deep learning
DNN	Deep neural networks
DRL	Deep reinforcement learning
DQN	Deep Q-learning
DDPG	Deep deterministic policy gradient
IoT	Internet of Things
MDP	Markov decision process
MEC	Multi-access edge computing
OO	Ordinal optimization
QoS	Quality of service
SDN	Software defined network
SAC	Soft actor-critic
TD3	Twin delayed deep deterministic policy gradient
VM	Virtual machine

Author Contributions

SQS had contributions to algorithm design, experimental simulations, and paper writing. YYW had contributions to theoretical design and analysis and paper writing. Both authors read and approved the final manuscript.

Funding

This study was partly supported by the Natural Science Foundation of China (NSFC) under Grant No. 62171046.

Data Availability

No available data because DRL agent learns from environment interactions.

Declarations

Conflict of interest

The authors declare that they have no conflict of interest.

Received: 25 February 2025 Accepted: 27 October 2025
Published online: 21 November 2025

References

1. K. Kaur, S. Garg, G.S. Aujla, N. Kumar, J.J.P.C. Rodrigues, M. Guizani, Edge computing in the industrial Internet of things environment: software-defined-networks-based Edge-Cloud interplay. IEEE Commun. Mag. **56**(2), 44–51 (2018)
2. J. Zhang, Z. Ning, M. Waqas, H. Alasmary, S. Tu, S. Chen, Hybrid Edge-Cloud collaborator resource scheduling approach based on deep reinforcement learning and multiobjective optimization. IEEE Trans. Comp. **73**(1), 192–205 (2024)
3. X. Chen, L. Jiao, W. Li, X. Fu, Efficient multi-user computation offloading for mobile edge cloud computing. IEEE/ACM Trans. Netw. **24**(5), 2795–2808 (2016)
4. W. Chen, Z. Zhang, Z. Hong et al., Cooperative and distributed computation offloading for blockchain-empowered industrial Internet of things. IEEE Internet Things J. **6**(5), 8433–8446 (2019)
5. M. Raeisi-Varzaneh, O. Dakkak, A. Habbal, B.-S. Kim, Resource scheduling in Edge computing: architecture, taxonomy, open issues and future research directions. IEEE Access. **11**(5), 25329–25350 (2023)
6. A. Feriani, E. Hossain, Single and multi-agent deep reinforcement learning for AI-enabled wireless networks: a tutorial. IEEE Commun. Surveys Tutorials. **23**(2), 1226–1252 (2021)
7. M.S. Munir et al., Risk-aware energy scheduling for edge computing with microgrid: a multi-agent deep reinforcement learning approach. IEEE Trans. Netw. Service Manag. **18**(3), 3476–3497 (2021)
8. Z. Ning et al., Deep reinforcement learning for intelligent Internet of Vehicles: an energy-efficient computational offloading scheme. IEEE Trans. Cogn. Commun. Netw. **5**(4), 1060–1072 (2019)
9. T.P. Lillicrap et al., Continuous control with deep reinforcement learning. (2016) https://arxiv.org/abs/1509.02971
10. W. Zhang et al., Deep reinforcement learning based resource management for DNN inference in industrial IoT. IEEE Trans. Veh. Technol. **70**(8), 7605–7618 (2021)
11. Y. Xu, L. Chen, Z. Lu, X. Du, J. Wu, P.C.K. Hung, An adaptive mechanism for dynamically collaborative computing power and task scheduling in Edge environment. IEEE Internet Things J. **10**(4), 3118–3129 (2023)
12. S. Lee, S. Lee, S. Lee, Deadline-aware task scheduling for IoT applications in collaborative Edge computing. IEEE Wirel. Commun. Lett. **10**(10), 2175–2179 (2021)
13. J. Lou, Z. Tang, S. Zhang, W. Jia, W. Zhao, J. Li, Cost-effective scheduling for dependent tasks with tight deadline constraints in mobile edge computing. IEEE Trans. Mobile Comput. **22**(10), 5829–5845 (2023)
14. H. Li, K.D.R. Assis, S. Yan, D. Simeonidou, DRL-based long-term resource planning for task offloading policies in multi-server Edge computing networks. IEEE Trans. Netw. Service Manag. **19**(4), 4151–4164 (2022)
15. Y. Ju et al., Joint secure offloading and resource allocation for vehicular edge computing network: a multi-agent deep reinforcement learning approach. IEEE Trans. Intell. Transp. Syst. **24**(5), 5555–5569 (2023)

16. X. Qiu, L. Liu, W. Chen, Z. Hong, Z. Zheng, Online deep reinforcement learning for computation offloading in blockchain-empowered mobile edge computing. IEEE Trans. Veh. Technol. **68**(8), 8050–8062 (2019)
17. J. Wang, L. Zhao, J. Liu, N. Kato, Smart resource allocation for mobile edge computing: a deep reinforcement learning approach. IEEE Trans. Emerg. Topics Comput. **9**(3), 1529–1541 (2021)
18. L. Ale, N. Zhang, X. Fang, X. Chen, S. Wu, L. Li, Delay-aware and energy-efficient computation offloading in mobile-edge computing using deep reinforcement learning. IEEE Trans. Cogn. Commun. Netw. **7**(3), 881–892 (2021)
19. Q. Liu, T. Xia, L. Cheng, M. van Eijk, T. Ozcelebi, Y. Mao, Deep reinforcement learning for load-balancing aware network control in IoT edge systems. IEEE Trans. Parallel Distrib. Syst. **33**(6), 1491–1502 (2022)
20. R. Xie, H. Zhu, Q. Tang, Q. Chen, S. Qiao, T. Huang, Joint task scheduling and load balancing in computing power network-enabled Edge computing systems, in Proceedings : 2022 IEEE 8th International Conference on Computer and Communications (ICCC) 563–568 (2022)
21. Q. Tang et al., Distributed task scheduling in serverless edge computing networks for the Internet of things: a learning approach. IEEE Internet Things J. **9**(20), 19634–19648 (2022)
22. R. Xie, D. Gu, Q. Tang, T. Huang, F.R. Yu, Workflow scheduling in serverless Edge computing for the industrial Internet of things: a learning approach. IEEE Trans. Indust. Inf. **19**(7), 8242–8252 (2023)
23. M. Akbari, M.R. Abedi, R. Joda et al., Age of information aware VNF scheduling in industrial IoT using deep reinforcement learning. IEEE J. Sel. Areas Commun. **39**(8), 2487–2500 (2021)
24. M. Pourghasemian, M.R. Abedi, S.S. Hosseini et al., AI-based mobility-aware energy efficient resource allocation and trajectory design for NFV enabled aerial networks. IEEE Trans. Green Commun. Netw. **7**(1), 281–297 (2023)
25. M.R. Abedi, N. Mokari, M.R. Javan et al., Low complexity and mobility-aware robust radio, storage, computing, and cost management for cellular vehicular networks. IEEE Trans. Vehicular Technol. **74**(2), 3327–3344 (2025)
26. Y. Ho, Q. Zhao, Q. Jia, Ordinal optimization: soft optimization for hard problems. New York, (2008)
27. T. Long, Q. Jia, G. Wang, Y. Yang, Efficient real-time EV charging scheduling via ordinal optimization. IEEE Trans. Smart Grid. **12**(5), 4029–4038 (2021)

Publisher's Note

Springer Nature remains neutral with regard to jurisdictional claims in published maps and institutional affiliations.